

SITUAÇÃO DE APRENDIZAGEM

Estratégia Desafiadora: Projeto Bolos do Jacquin

Mundo Comum

Jacquin é um microempreendedor da cidade de São Paulo que trabalha sozinho na produção de doces, principalmente bolos. Seu catálogo inclui produtos de chocolate, frutas, macarons e outras categorias que poderão ser ampliadas conforme o crescimento do negócio.

Atualmente, a divulgação acontece principalmente pelo boca a boca. Embora as indicações ajudem a conquistar clientes, esse formato limita o alcance do empreendimento e dificulta que novas pessoas conheçam, de maneira organizada, a variedade, os preços e as características dos produtos.

Para ampliar o alcance do negócio e organizar a divulgação dos produtos, Jacquin precisa de um sistema capaz de reunir as informações do catálogo, direcionar os interessados ao WhatsApp, proteger os dados, aplicar regras de negócio e manter os registros de forma consistente.

Jacquin também percebeu que as opiniões dos clientes poderiam ajudar outras pessoas a comparar produtos e tomar decisões. Para isso, o catálogo deverá apresentar avaliações registradas por usuários identificados, sem transformar a aplicação em uma loja virtual ou substituir o atendimento realizado pelo WhatsApp.

Chamado para a aventura

Jacquin contratou você para desenvolver individualmente o sistema web Bolos do Jacquin. A solução deverá integrar uma interface em React com TypeScript a uma API REST desenvolvida em C# com ASP.NET Core, oferecendo catálogo público, autenticação, persistência em banco de dados e um sistema de avaliações.

O sistema deverá permitir que visitantes conheçam os produtos e iniciem o atendimento pelo WhatsApp, que clientes cadastrados avaliem os itens e que Jacquin administre produtos, categorias, usuários e avaliações em uma área protegida.

Sistema Web

Objetivo geral

Desenvolver uma aplicação web responsiva, integrada e segura para divulgar os produtos do Bolos do Jacquin, ampliar o alcance do microempreendimento, registrar avaliações de clientes e disponibilizar uma gestão administrativa apoiada por uma API REST em ASP.NET Core.

Perfis de usuário

1. Administrador: perfil utilizado por Jacquin para gerenciar produtos, categorias, usuários e avaliações, além de acompanhar informações gerais do catálogo.
2. Cliente: usuário cadastrado que poderá utilizar os recursos públicos do catálogo e registrar, editar ou excluir suas próprias avaliações.
3. Visitante: usuário não autenticado que poderá consultar o catálogo, pesquisar, aplicar filtros, visualizar detalhes e iniciar o atendimento pelo WhatsApp.

Funcionalidades essenciais

1. Disponibilizar publicamente o catálogo de produtos, sem exigir autenticação para consulta.
2. Permitir pesquisa por nome e filtros por categoria e faixa de preço.
3. Exibir uma página de detalhes com informações completas, disponibilidade, média das notas e avaliações publicadas.
4. Direcionar o visitante ao número de WhatsApp configurado para o atendimento comercial.
5. Permitir cadastro e autenticação de clientes por meio da API REST.
6. Utilizar JWT para controlar o acesso às rotas e às funcionalidades protegidas.
7. Permitir que Jacquin cadastre, consulte, edite, desative e, quando permitido, exclua produtos e categorias.
8. Permitir que clientes autenticados atribuam notas de uma a cinco estrelas e publiquem comentários opcionais.
9. Garantir que cada cliente possua apenas uma avaliação por produto, com possibilidade de edição ou exclusão da própria avaliação.
10. Calcular e apresentar a média e a quantidade de avaliações publicadas de cada produto.
11. Permitir que o administrador oculte ou restaure avaliações, preservando os registros necessários para consulta.
12. Persistir os dados em banco relacional e integrar front-end e backend por meio de requisições HTTP com Axios.

CATÁLOGO PÚBLICO E CONTATO

O catálogo público será o principal ponto de contato entre o Bolos do Jacquin e seus potenciais clientes. A aplicação deverá combinar informações comerciais, disponibilidade e avaliações para ajudar o visitante a conhecer as opções antes de conversar com Jacquin.

Objetivo funcional

Oferecer uma experiência de consulta clara, responsiva e confiável, permitindo que o visitante encontre produtos, compare opções, leia avaliações e chegue ao atendimento pelo WhatsApp mais decidido sobre o que deseja pedir.

Requisitos

- A página principal deverá apresentar os produtos ativos em cards com imagem, nome, preço, categoria, descrição curta, disponibilidade, média de avaliações e quantidade de avaliações publicadas.
- A pesquisa deverá localizar produtos pelo nome, desconsiderando diferenças entre letras maiúsculas e minúsculas.
- Os filtros por categoria e faixa de preço deverão funcionar isoladamente ou em conjunto com a pesquisa.
- A página de detalhes deverá apresentar imagem, nome, preço, categoria, descrições curta e longa, disponibilidade, média das notas e lista de avaliações publicadas.
- Produtos indisponíveis deverão permanecer consultáveis e ser identificados visualmente, sem perder seu histórico de avaliações.
- A ausência de avaliações deverá ser informada de forma compreensível, sem exibir uma média artificial.

- O botão de WhatsApp deverá permanecer visível na página principal e utilizar o número configurado na aplicação.
- O redirecionamento ao WhatsApp não deverá preencher automaticamente uma mensagem sobre um produto.
- A interface deverá tratar carregamento, erro de comunicação, ausência de dados e pesquisa sem resultados.
- Todas as telas públicas deverão permanecer utilizáveis em computadores, tablets e celulares.

Fluxo principal do visitante

1. O visitante acessa o catálogo e visualiza os produtos disponíveis.
2. Utiliza a pesquisa ou os filtros para reduzir as opções apresentadas.
3. Seleciona um produto, consulta suas informações e lê as avaliações publicadas.
4. Retorna à tela principal e utiliza o botão de WhatsApp para iniciar o atendimento com Jacquin.
5. A negociação, a definição do pedido e o pagamento acontecem fora do sistema.

Entregáveis

- Página inicial com catálogo, pesquisa, filtros, indicadores de avaliação e botão de WhatsApp.
- Página de detalhes com dados completos do produto e avaliações publicadas.
- Componentes reutilizáveis para cards, filtros, notas, comentários e estados da interface.
- Integração funcional com os endpoints públicos da API REST.
- Evidências dos principais fluxos em diferentes tamanhos de tela.

Critérios de conclusão

- Os produtos retornados pela API são apresentados corretamente no catálogo.
- Pesquisa e filtros produzem resultados coerentes quando utilizados separadamente ou em conjunto.
- A média e a quantidade de avaliações correspondem apenas às avaliações publicadas.
- A página de detalhes corresponde ao produto selecionado e trata produtos inexistentes.
- O botão de WhatsApp abre o canal de atendimento configurado.
- A interface permanece clara, legível e funcional em diferentes dispositivos.

AUTENTICAÇÃO E ÁREA ADMINISTRATIVA

O sistema deverá diferenciar visitantes, clientes e administrador. O backend será responsável por autenticar os usuários, emitir tokens e verificar as permissões antes de executar qualquer operação protegida.

Objetivo funcional

Permitir que clientes acessem recursos pessoais e que Jacquin mantenha o catálogo e seus registros por meio de uma área administrativa protegida, com validações no front-end e no backend.

Autenticação e autorização

- Disponibilizar cadastro de clientes com nome, e-mail e senha.

- Impedir o cadastro de e-mails duplicados.
- Armazenar somente o hash da senha, nunca a senha em texto puro.
- Disponibilizar login para clientes e administrador.
- Emitir um token JWT contendo a identificação e o perfil do usuário autenticado.
- Proteger os endpoints e as rotas administrativas com autorização baseada em perfil.
- Permitir que o cliente consulte seus dados básicos e suas próprias avaliações.
- Permitir que o administrador consulte usuários e desative contas quando necessário, sem apagar avaliações relacionadas.

Gestão de produtos e categorias

- Permitir o cadastro de produtos com nome, preço, endereço da imagem, categoria, descrição curta, descrição longa e disponibilidade.
- Permitir a consulta, a edição e a alteração da disponibilidade dos produtos.
- Utilizar um endereço de imagem informado no formulário, sem realizar upload real de arquivos.
- Priorizar a desativação de produtos para preservar vínculos e históricos.
- Permitir a exclusão física apenas quando não houver avaliações ou outros registros dependentes.
- Permitir o cadastro, a consulta e a edição de categorias.
- Impedir a exclusão de uma categoria enquanto existirem produtos associados a ela.
- Apresentar confirmações para ações destrutivas e mensagens claras para sucesso ou erro.
- Atualizar as listagens após as operações sem exigir recarregamento manual da página.

Gestão de avaliações

- Permitir que o administrador consulte avaliações por produto, cliente, nota e situação.
- Permitir que avaliações publicadas sejam ocultadas e que avaliações ocultas sejam restauradas.
- Registrar o motivo da ocultação para fins de rastreabilidade.
- Impedir que o administrador altere a nota ou o texto escrito pelo cliente.

Entregáveis

- Telas de cadastro, login e controle da sessão autenticada.
- Painel administrativo com gestão de produtos, categorias, usuários e avaliações.
- Formulários de cadastro e edição com validações.
- Proteção de rotas e componentes conforme o perfil autenticado.
- Evidências dos fluxos autorizados, não autorizados e proibidos.

Critérios de conclusão

- Tokens válidos permitem acesso somente aos recursos compatíveis com o perfil do usuário.
- Acesso sem token, com token inválido ou com perfil insuficiente é rejeitado corretamente.
- As alterações administrativas são persistidas e refletidas no catálogo público.
- Regras de vínculos impedem exclusões que deixariam dados inconsistentes.

- As mensagens da interface e da API informam claramente o resultado das operações.

SISTEMA DE AVALIAÇÕES

O sistema de avaliações permitirá que clientes identificados compartilhem sua opinião sobre os produtos. Essas informações serão exibidas no catálogo como apoio à decisão de outros usuários, mas não representarão uma compra verificada, pois o pedido e o pagamento continuarão acontecendo pelo WhatsApp.

Objetivo funcional

Registrar avaliações vinculadas a clientes e produtos, calcular indicadores confiáveis e permitir o controle do próprio conteúdo pelo cliente e da visibilidade pelo administrador.

Regras de negócio

- Somente clientes autenticados poderão registrar avaliações.
- Cada avaliação deverá possuir uma nota inteira de uma a cinco estrelas e poderá possuir um comentário opcional.
- Quando informado, o comentário deverá respeitar os limites de tamanho definidos pela aplicação.
- Cada cliente poderá manter apenas uma avaliação por produto.
- Uma nova tentativa de avaliar o mesmo produto deverá retornar conflito e orientar o cliente a editar a avaliação existente.
- O cliente poderá editar a nota e o comentário de sua própria avaliação.
- O cliente poderá excluir sua própria avaliação mediante confirmação.
- O backend deverá obter o identificador do cliente a partir do token, sem confiar em um identificador enviado livremente pelo front-end.
- A avaliação deverá registrar data de criação, data da última alteração e situação PUBLICADA ou OCULTA.
- Somente avaliações PUBLICADAS deverão aparecer no catálogo e participar do cálculo da média.
- A ocultação administrativa deverá preservar a avaliação e registrar o motivo da decisão.
- O sistema não deverá utilizar a expressão compra verificada, pois não existe controle de pedidos dentro da aplicação.

Fluxo principal do cliente

1. O cliente cria uma conta ou realiza login.
2. Acessa a página de detalhes de um produto.
3. Informa uma nota e, opcionalmente, um comentário.
4. A API valida a autenticação, o produto, a nota e a inexistência de outra avaliação do mesmo cliente para o produto.
5. A avaliação é persistida e a média do produto é atualizada na próxima consulta.
6. O cliente poderá editar ou excluir a avaliação posteriormente.

Entregáveis

- Componente visual de estrelas para exibição e seleção de notas.

- Formulário de criação e edição de avaliação.
- Listagem de avaliações na página do produto.
- Área do cliente para consultar e gerenciar as próprias avaliações.
- Área administrativa para ocultar e restaurar avaliações.
- Endpoints e regras de banco responsáveis por impedir avaliações duplicadas.

Critérios de conclusão

- Usuários não autenticados não conseguem criar, editar ou excluir avaliações.
- Um cliente não consegue alterar ou excluir a avaliação de outro cliente.
- Notas fora do intervalo definido e avaliações duplicadas são rejeitadas.
- A média é recalculada corretamente após criação, edição, exclusão, ocultação ou restauração.
- Avaliações ocultas permanecem registradas, mas não aparecem publicamente nem afetam a média.

API REST E BANCO DE DADOS

A API REST será a camada responsável por aplicar as regras de negócio, proteger os recursos e persistir os dados. O front-end não deverá acessar o banco diretamente nem decidir sozinho sobre autenticação, autorização ou integridade das avaliações.

Tecnologias definidas

C# com ASP.NET Core para a API REST, Entity Framework Core para acesso aos dados, SQL Server como banco relacional, JWT para autenticação, Swagger/OpenAPI para documentação, React com TypeScript no front-end e Axios para comunicação HTTP.

Organização da API

- Organizar o código por responsabilidades, separando entidades, DTOs, contexto de dados, serviços, controladores e configurações.
- Utilizar DTOs específicos para entrada e saída, evitando expor diretamente as entidades do banco.
- Utilizar operações assíncronas nas consultas e alterações de dados.
- Validar dados de entrada no backend, independentemente das validações existentes no front-end.
- Retornar códigos HTTP coerentes, incluindo 200, 201, 204, 400, 401, 403, 404 e 409 conforme o resultado da operação.
- Implementar tratamento centralizado de exceções e respostas de erro consistentes.
- Configurar CORS somente para as origens necessárias ao desenvolvimento da interface.
- Documentar rotas, parâmetros, corpos e respostas com Swagger/OpenAPI.
- Permitir paginação e filtros nos endpoints de produtos, usuários e avaliações quando houver grande volume de registros.

Endpoints mínimos

- Autenticação: cadastro de cliente, login e consulta do usuário autenticado.
- Produtos: listagem pública, consulta por identificador e operações administrativas de cadastro, edição, disponibilidade e exclusão permitida.

- Categorias: listagem pública e operações administrativas de manutenção.
- Avaliações: listagem pública por produto, criação autenticada, edição e exclusão pelo proprietário.
- Moderação: consulta administrativa e alteração da situação da avaliação.
- Usuários: consulta e alteração de situação pelo administrador.

Estrutura mínima dos dados

- Usuário: identificador, nome, e-mail único, hash da senha, perfil, situação e data de cadastro.
- Categoria: identificador e nome único.
- Produto: identificador, nome, preço, endereço da imagem, categoria, descrição curta, descrição longa, disponibilidade e situação.
- Avaliação: identificador, nota, comentário, situação, motivo da ocultação, datas de criação e alteração, usuário e produto.
- Relacionamentos: categoria possui vários produtos; usuário possui várias avaliações; produto possui várias avaliações.
- Restrição única: combinação entre usuário e produto na tabela de avaliações.

Entregáveis

- Projeto ASP.NET Core com código-fonte organizado e configurações documentadas.
- Banco SQL Server criado por migrations ou scripts reproduzíveis.
- Documentação navegável no Swagger/OpenAPI.
- Conjunto mínimo de dados para demonstração.
- Evidências das requisições e respostas dos principais endpoints.

Critérios de conclusão

- A API inicializa e se conecta ao banco sem depender de alterações manuais não documentadas.
- Relacionamentos e restrições impedem registros órfãos ou avaliações duplicadas.
- Endpoints públicos e protegidos respeitam as permissões definidas.
- Erros de validação e conflitos retornam mensagens e códigos HTTP coerentes.
- A documentação permite que outra pessoa compreenda e teste os recursos disponíveis.

FRONT-END E INTEGRAÇÃO

A interface será desenvolvida em React com TypeScript e consumirá exclusivamente os recursos disponibilizados pela API. A aplicação deverá oferecer uma experiência simples e acessível, integrando autenticação, avaliações e gestão administrativa sem comprometer a responsividade.

Objetivo funcional

Construir uma interface tipada, acessível e reutilizável que represente corretamente os perfis de usuário, os estados da API e as regras do catálogo e das avaliações.

Requisitos

- Organizar a aplicação em componentes, páginas, rotas, serviços, contextos ou stores, tipos e recursos visuais.
- Criar telas públicas de catálogo, detalhes do produto, cadastro e login.
- Criar área do cliente para consulta e manutenção das próprias avaliações.
- Criar área administrativa para produtos, categorias, usuários e avaliações.
- Proteger rotas e elementos da interface conforme a autenticação e o perfil.
- Centralizar a configuração da URL da API e do número de WhatsApp.
- Utilizar Axios para as requisições e um interceptor para incluir o token nas operações protegidas.
- Criar tipos TypeScript para usuários, produtos, categorias, avaliações, filtros e respostas de erro.
- Validar formulários e apresentar mensagens compreensíveis junto aos campos correspondentes.
- Tratar carregamento, sucesso, erro, ausência de dados, acesso negado e sessão expirada.
- Aplicar boas práticas de responsividade, navegação por teclado, contraste, rótulos e textos alternativos.
- Não depender de uma biblioteca visual obrigatória; a escolha deverá ser documentada caso alguma seja utilizada.

Entregáveis

- Aplicação React com TypeScript integrada à API REST.
- Serviços de comunicação com os endpoints de autenticação, produtos, categorias, avaliações e usuários.
- Componentes reutilizáveis de cards, estrelas, formulários, alertas, confirmações e estados de carregamento.
- Rotas públicas, autenticadas e administrativas.
- Evidências dos fluxos completos no navegador.

Critérios de conclusão

- A interface apresenta somente ações permitidas para o perfil autenticado.
- O token é enviado corretamente e a aplicação reage a sessões inválidas ou expiradas.
- Alterações realizadas no front-end são persistidas pela API e refletidas nas consultas seguintes.
- Formulários impedem envios inválidos e também exibem os erros retornados pelo backend.
- A aplicação permanece utilizável em diferentes tamanhos de tela.

SEGURANÇA, TESTES E LIMITES DO PROJETO

A conclusão da solução dependerá não apenas da existência das telas e endpoints, mas também da proteção dos dados, da documentação das decisões e da execução de testes que demonstrem o comportamento esperado nos principais cenários.

Regras de segurança

- Senhas deverão ser armazenadas por meio de hash adequado e nunca retornadas pela API.
- A chave de assinatura do JWT, strings de conexão e demais segredos deverão permanecer em variáveis de ambiente ou mecanismos equivalentes, fora do repositório.

- Toda autorização deverá ser validada no backend, mesmo quando a interface esconder botões ou rotas.
- A API deverá validar identificadores, limites de texto, notas, preços e demais dados recebidos.
- O acesso ao banco deverá ocorrer exclusivamente pelo backend por meio do Entity Framework Core.
- Mensagens de erro não deverão expor stack traces, credenciais ou detalhes internos em respostas de produção.
- O repositório não deverá conter senhas reais, tokens válidos ou configurações sensíveis.

Testes e evidências

- Implementar testes automatizados dos principais serviços ou endpoints da API utilizando uma ferramenta compatível com .NET, como xUnit.
- Testar cadastro, login, token inválido, acesso sem permissão e usuário desativado.
- Testar cadastro, edição, desativação e regras de exclusão de produtos e categorias.
- Testar criação, edição, exclusão, duplicidade, ocultação e restauração de avaliações.
- Testar o cálculo da média antes e depois das alterações nas avaliações.
- Testar respostas para recursos inexistentes, dados inválidos, conflitos e falhas de comunicação.
- Registrar evidências dos fluxos executados no Swagger e no navegador, com o resultado esperado e o resultado obtido.
- Garantir que os testes possam ser repetidos por outra pessoa a partir das instruções fornecidas.

Limites do projeto

- A aplicação não terá carrinho, checkout, pagamento, controle de pedidos ou acompanhamento de entrega.
- A negociação comercial acontecerá diretamente pelo WhatsApp.
- As avaliações não serão classificadas como compras verificadas, pois o sistema não registra a conclusão das vendas.
- As imagens serão representadas por endereços informados nos formulários; não haverá armazenamento real de arquivos.
- Não será exigida publicação em ambiente de produção.
- O projeto deverá ser desenvolvido individualmente como Situação de Aprendizagem.

Entregáveis finais

- Código-fonte do front-end em React com TypeScript.
- Código-fonte da API REST em C# com ASP.NET Core.
- Banco de dados, migrations ou scripts e dados mínimos para demonstração.
- Documentação da API no Swagger/OpenAPI.
- Instruções para configurar o banco, as variáveis de ambiente e executar front-end e backend.
- Registro das decisões técnicas, regras de negócio, testes executados e evidências obtidas.

Resultado esperado

Ao final da Situação de Aprendizagem, a aplicação deverá demonstrar integração entre front-end, API REST e banco de dados, controle de acesso por perfil e aplicação consistente das regras do sistema de avaliações.

O visitante deverá conseguir conhecer os produtos e chegar ao atendimento com mais informações. O cliente cadastrado deverá participar do catálogo por meio de avaliações próprias, enquanto Jacquín deverá manter os dados e a visibilidade dos conteúdos por uma área administrativa protegida.

A solução deverá evidenciar que as decisões importantes permanecem no backend, que os dados podem ser reproduzidos em outro ambiente e que os fluxos foram testados e documentados de forma compreensível.

Critérios finais de conclusão

- O catálogo público, a autenticação, a área do cliente, a área administrativa e as avaliações funcionam de forma integrada.
- A API aplica as regras de negócio e mantém os dados consistentes no banco relacional.
- As permissões impedem acessos e alterações indevidas.
- A média das avaliações permanece correta em todos os fluxos previstos.
- O projeto pode ser configurado e executado por outra pessoa a partir da documentação entregue.
- A interface é responsiva, o código está organizado e as evidências demonstram os resultados obtidos.